ASSIGNMENT 3 – CO3

CO3 Coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 07/11/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment3 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: **https://github.com/knishalini1497/DSA0101-OOPS/ASSIGNMENTS at main · knishalini1497/DSA0101-OOPS**

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

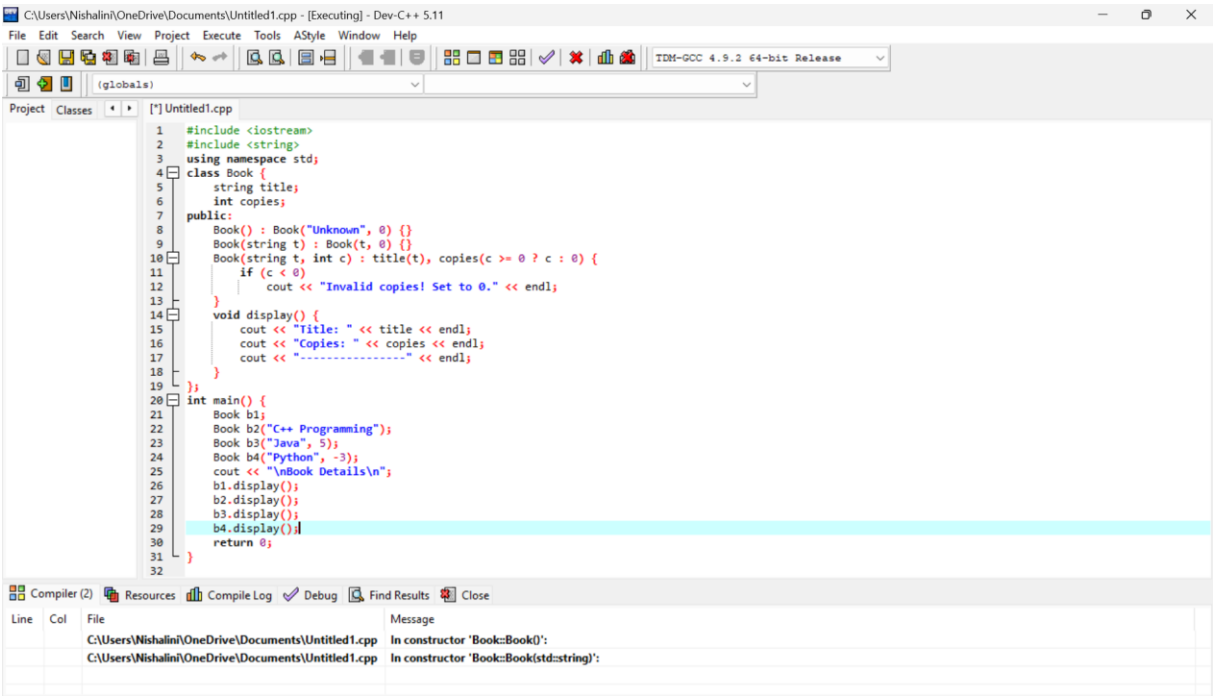
Q1 – Constructors and Constructor Overloading

Bloom's Level: K5 – Evaluate

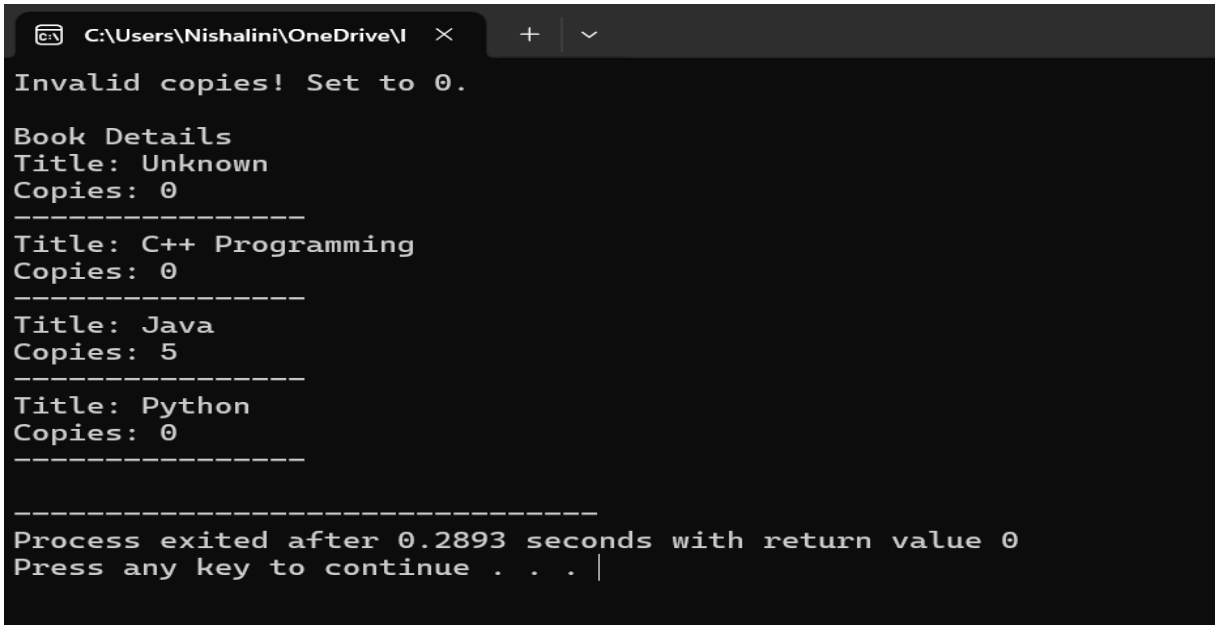
Problem Context – Library Book Records

A library creates Book objects in several situations: a new book with full details, a donated book with only a title, and an empty record that will be filled later. Some records were created with garbage values in the number of copies.

1.program



2. Output



3.Design & Analysis

- **When does the compiler supply a default constructor?** Only when the class declares no constructor at all. As soon as any constructor is written (even a parameterised one),

the compiler does not generate Book(), so Book b; and Book shelf[3]; stop compiling unless you write one (or use = default).

- **Why initializer lists?** Members are initialised directly with the right value instead of being default-constructed and then assigned. This is mandatory for const members, reference members and members without a default constructor, and it is faster for objects such as std::string.
- **Order of initialisation:** members are initialised in the order of their declaration in the class, not the order written in the list. Writing them in a different order only produces the -Wreorder warning and can cause use of an uninitialised member (shown below).

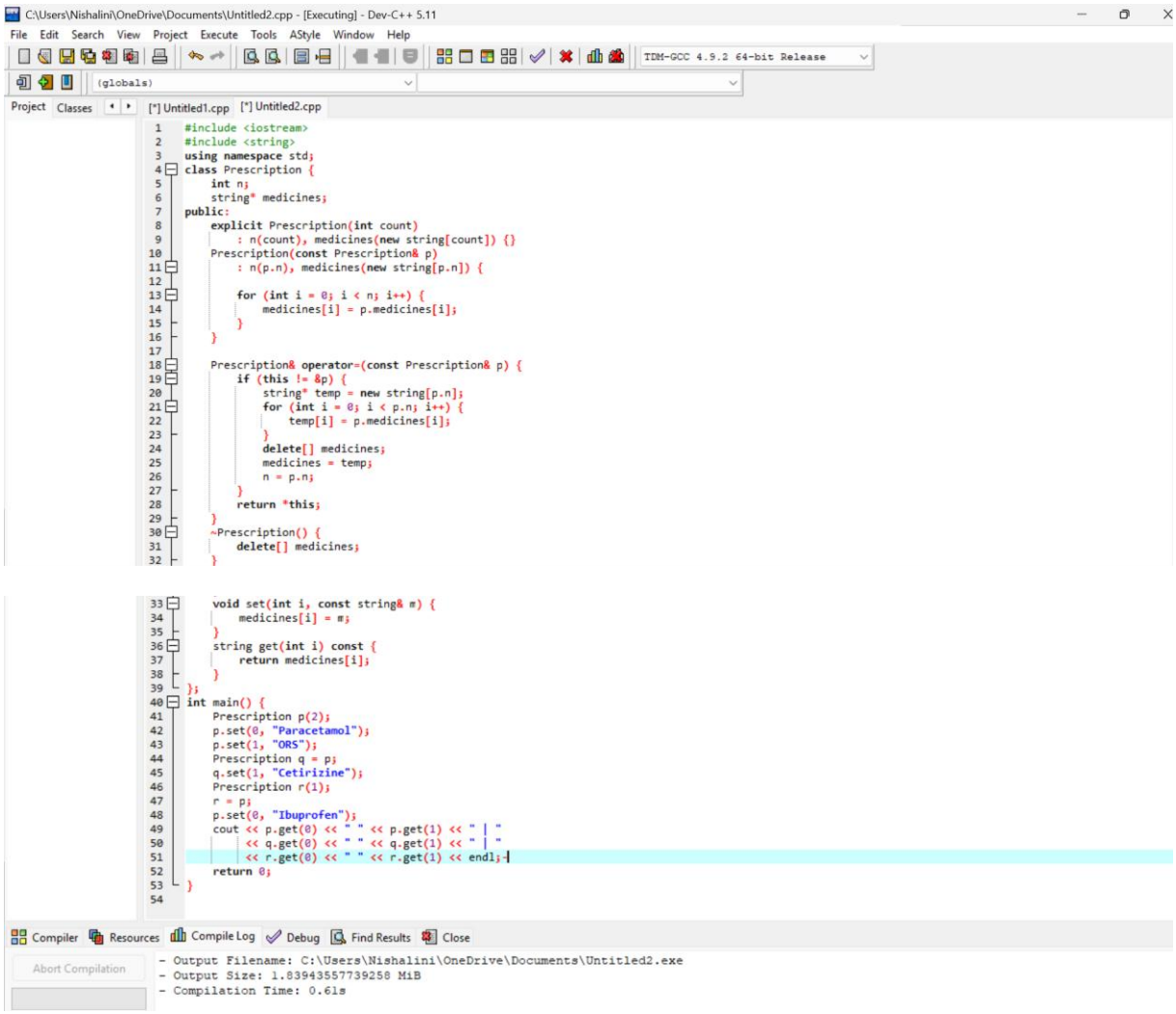
Q2 – Copy Constructors: Shallow and Deep Copies

Bloom's Level: K5 – Evaluate

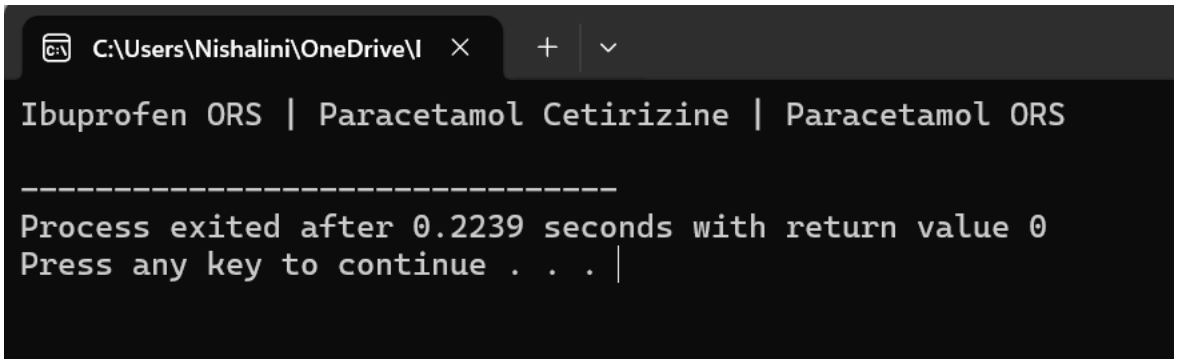
Problem Context – Hospital Prescription Records

A Prescription object holds a dynamically allocated array of medicine names. When a doctor copies a prescription to modify it for a follow-up visit, the original prescription also changes, and the program sometimes crashes when it ends.

1. Program



2.output



3.Memory Diagram – Deep Copy

```
p  —> [Ibuprofen] [ORS]

q  —> [Paracetamol] [Cetirizine]

r  —> [Paracetamol] [ORS]
```

Each object has its own separate memory.

4. Shallow Copy vs Deep Copy

Shallow Copy	Deep Copy
Copies memory address	Copies actual data
Shares same array	Creates a new array
Changes affect both objects	Objects are independent
May cause double deletion	Prevents double deletion

5. Rule of Three

The Rule of Three contains:

1. Copy constructor
2. Copy assignment operator
3. Destructor

These three functions are required when a class manages dynamic memory.

6. When is a Copy Constructor Called?

1. When one object is initialized using another object.
2. When an object is passed to a function by value.

3. When an object is returned by value (a copy may occur, but compilers often apply copy elision).

7. Self-Assignment

```
p = p;
```

The condition:

```
if (this != &p)
```

prevents an object from copying itself unnecessarily.

8. Time and Space Complexity

Medicines	Copy Time	Extra Space
10	O(n)	O(n)
1,000	O(n)	O(n)
100,000	O(n)	O(n)

9. Conclusion

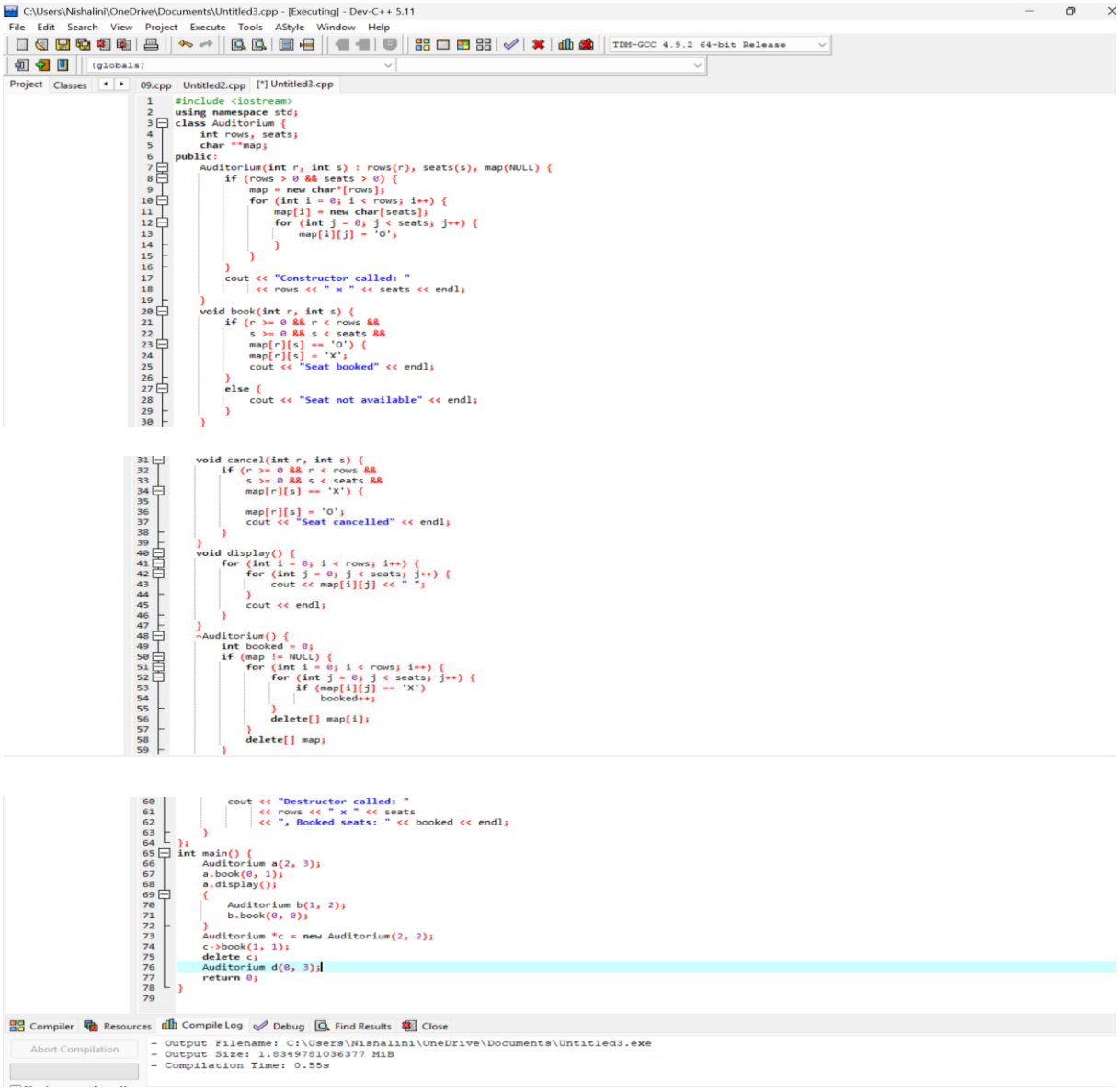
Deep copying creates separate memory for each object. The copy constructor, copy assignment operator and destructor prevent shared-memory problems and double deletion.

Result: Thus, the program successfully implements deep copying and produces the expected output.

Q3 – Dynamic Constructors and Destructors
Bloom's Level: K5 – Evaluate
Problem Context – Auditorium Seat Booking

An auditorium's seat map is created when an event is set up; the number of rows and seats per row differs from event to event. When the event object is destroyed, all memory must be released and a closing summary written.

1.Program



```
#include <iostream>
using namespace std;
class Auditorium {
    int rows, seats;
    char **map;
public:
    Auditorium(int r, int s) : rows(r), seats(s), map(NULL) {
        if (rows > 0 && seats > 0) {
            map = new char*[rows];
            for (int i = 0; i < rows; i++) {
                map[i] = new char[seats];
                for (int j = 0; j < seats; j++) {
                    map[i][j] = 'O';
                }
            }
        }
        cout << "Constructor called: "
             << rows << " x " << seats << endl;
    }
    void book(int r, int s) {
        if (r >= 0 && r < rows &&
            s >= 0 && s < seats &&
            map[r][s] == 'O') {
            map[r][s] = 'X';
            cout << "Seat booked" << endl;
        }
        else {
            cout << "Seat not available" << endl;
        }
    }

    void cancel(int r, int s) {
        if (r >= 0 && r < rows &&
            s >= 0 && s < seats &&
            map[r][s] == 'X') {

            map[r][s] = 'O';
            cout << "Seat cancelled" << endl;
        }
    }
    void display() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < seats; j++) {
                cout << map[i][j] << " ";
            }
            cout << endl;
        }
    }
    ~Auditorium() {
        int booked = 0;
        if (map != NULL) {
            for (int i = 0; i < rows; i++) {
                for (int j = 0; j < seats; j++) {
                    if (map[i][j] == 'X')
                        booked++;
                }
                delete[] map[i];
            }
            delete[] map;
        }

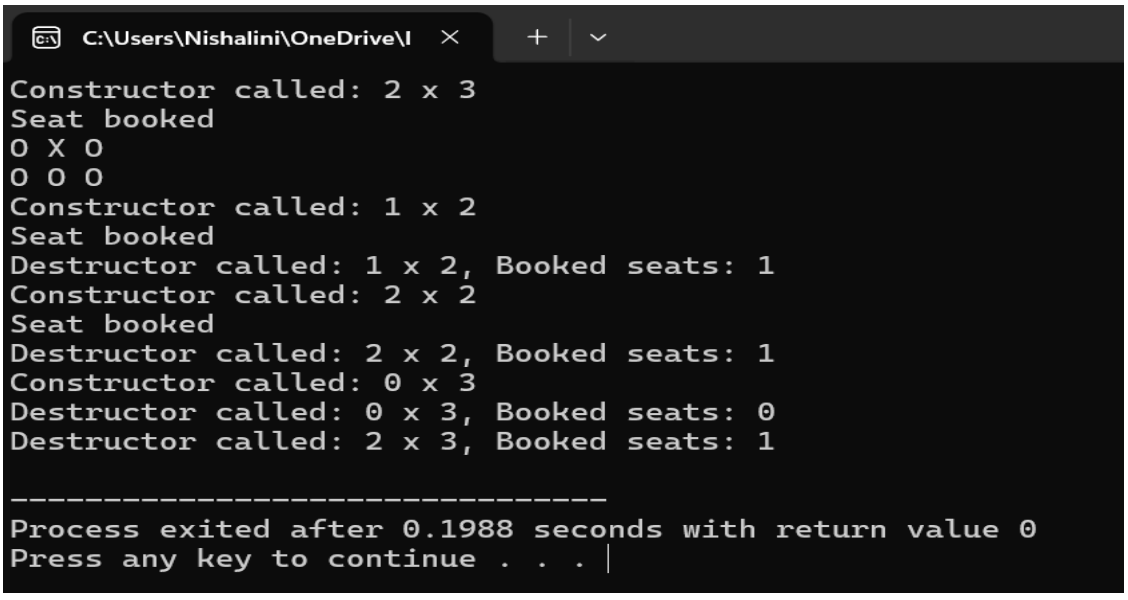
        cout << "Destructor called: "
             << rows << " x " << seats
             << ", Booked seats: " << booked << endl;
    }
};
int main() {
    Auditorium a(2, 3);
    a.book(0, 1);
    a.display();
    {
        Auditorium b(1, 2);
        b.book(0, 0);
    }
    Auditorium *c = new Auditorium(2, 2);
    c->book(1, 1);
    delete c;
    Auditorium d(0, 3);
    return 0;
}
```

Compiler | Resources | Compile Log | Debug | Find Results | Close

Abort Compilation

- Output Filename: C:\Users\Nishalini\OneDrive\Documents\Untitled3.exe
- Output Size: 1.8349781036377 MiB
- Compilation Time: 0.55s

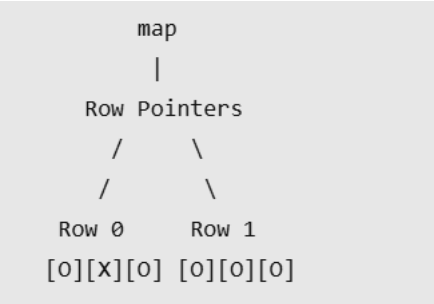
2.Output



```
C:\Users\Nishalini\OneDrive\I
Constructor called: 2 x 3
Seat booked
O X O
O O O
Constructor called: 1 x 2
Seat booked
Destructor called: 1 x 2, Booked seats: 1
Constructor called: 2 x 2
Seat booked
Destructor called: 2 x 2, Booked seats: 1
Constructor called: 0 x 3
Destructor called: 0 x 3, Booked seats: 0
Destructor called: 2 x 3, Booked seats: 1

--------------------------------
Process exited after 0.1988 seconds with return value 0
Press any key to continue . . .
```

3. Memory Layout of Seat Map



- new char*[rows] allocates row pointers.
- new char[seats] allocates each row.
- delete[] map[i] deletes each row.
- delete[] map deletes the row pointers.

4. Constructor and Destructor Order

Object	Constructor	Destructor
Stack object	When created	At the end of its scope
Block object	When block starts	When block ends
Heap object	When new is used	When delete is used

Important: Local objects are destroyed in reverse order of their construction.

5. Important Questions

1. What happens if delete is forgotten?

Memory leak occurs because allocated memory is not released.

2. What happens if delete is used instead of delete[]?

It causes undefined behaviour when memory was allocated using new[].

3. Why does a destructor take no arguments?

- Destructor is automatically called when an object is destroyed.
- It has no return type.
- It takes no arguments.
- It cannot be overloaded.

4. What happens if rows = 0?

No memory is allocated for the seat map, and the destructor safely releases the null pointer.

6. Memory Calculation

Assume each seat occupies 1 byte.

Seat Map	Memory Required
10 × 20	200 bytes
100 × 200	20,000 bytes
1,000 × 2,000	2,000,000 bytes

These values exclude row pointers and allocation overhead.

7. RAII Concept

RAII means Resource Acquisition Is Initialization.

- Constructor allocates memory.
- Destructor releases memory automatically.
- Prevents memory leaks.
- Makes the program safer.

8. Predict the Output – Logger Program

Correct predicted output:

```
+A +B -B +Ac copyAc use -Ac +P -P end -A
```

Explanation

Statement	Output	Explanation
Logger a("A")	+A	Object A created
Logger b("B")	+B	Object B created
End of block	-B	Object B destroyed
use(a)	+Ac	Copy of A created
Copy constructor	copyAc	Copy message printed
Inside use()	use	Function executed
Function ends	-Ac	Copied object destroyed
new Logger("P")	+P	Heap object created

delete p	-P	Heap object destroyed
cout << "end "	end	End message printed
End of main	-A	Object A destroyed

Why is a copy made?

The function is:

```
void use(Logger)
```

It accepts the object by value. Therefore, a copy of a is created. The copied object is destroyed when the function finishes.

9. Alternative Concept – Single Array and Vector

Method 1: Single memory block

```
char* map = new char[rows * seats];

// Access seat
map[r * seats + s];

// Release memory
delete[] map;
```

Method 2: Using vector

```
#include <vector>

vector<vector<char>> map(
    rows, vector<char>(seats, 'O')
);
```

10. Complexity Analysis

Feature	2D Dynamic Array	Single Array	Vector
Access time	O(1)	O(1)	O(1)
Allocations	rows + 1	1	Typically rows + 1
Extra row pointers	Yes	No	Managed internally
Memory release	Manual	Manual	Automatic

- 2D array: Requires one allocation for row pointers and one for each row.
- Single array: Requires only one allocation and no row-pointer array.

- Vector: Manages memory automatically, with separate row storage in the nested-vector approach.

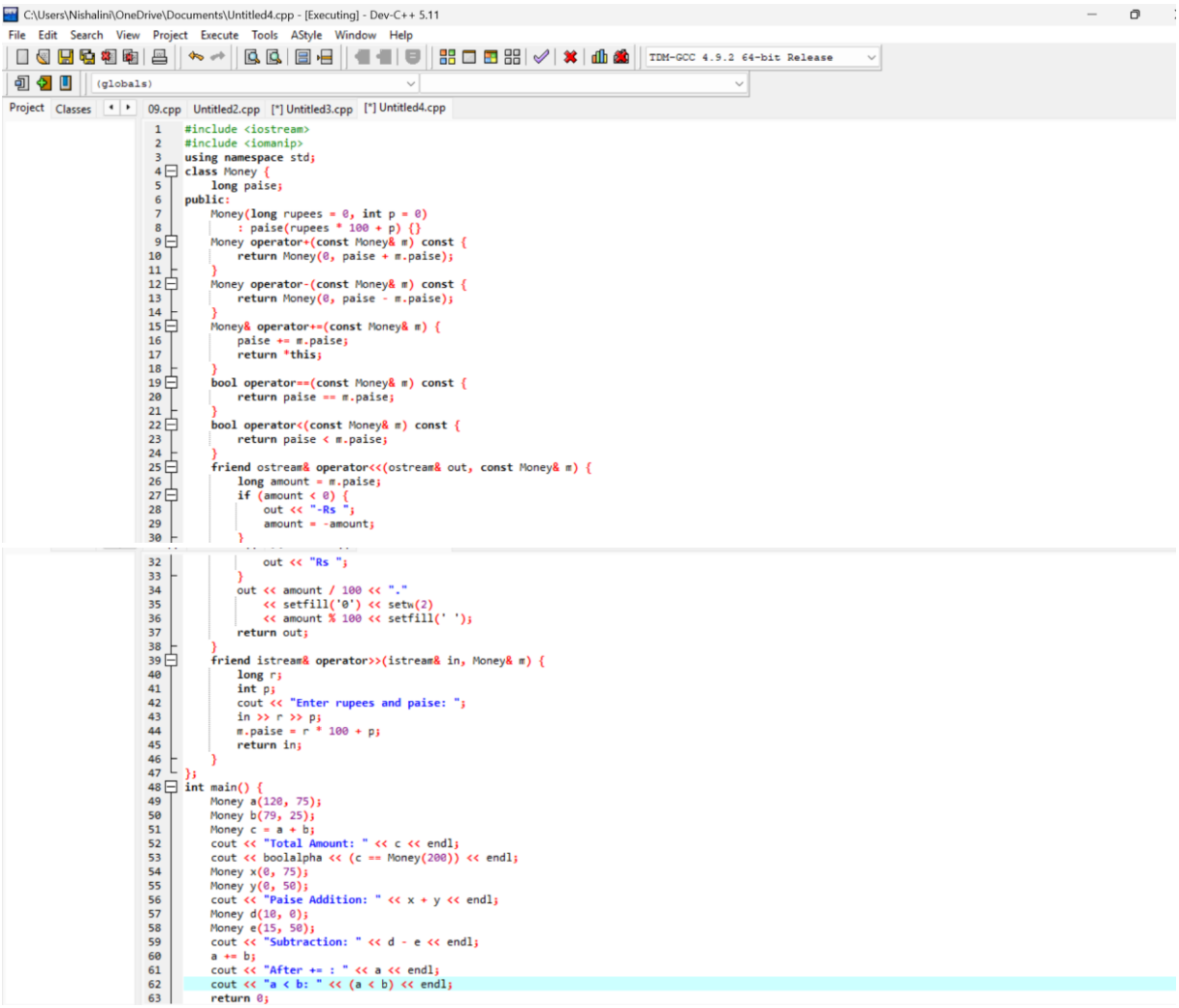
Q4 – Overloading Arithmetic and Stream Operators

Bloom's Level: K5 – Evaluate

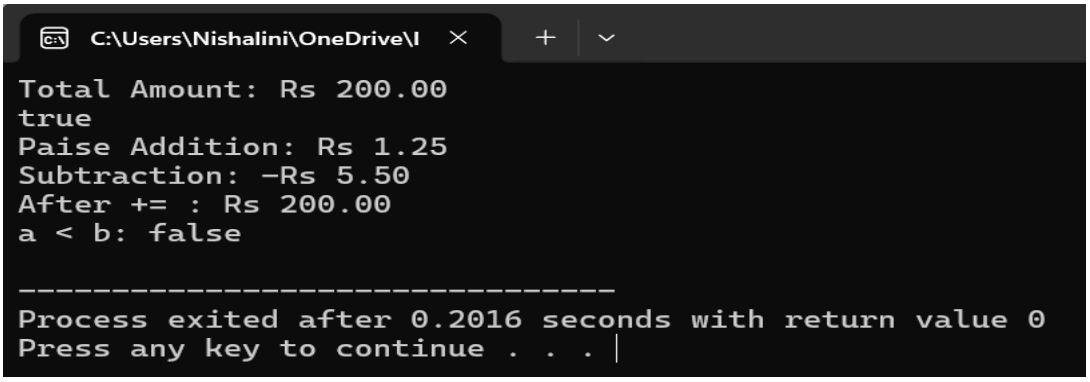
Problem Context – Money Arithmetic for a Billing Counter

A billing counter adds, subtracts and compares amounts of money stored in rupees and paise. The present code calls functions such as add(a, b) and print(c), which makes the bill calculation hard to read.

1.program



2.output



3. Explanation of Operators

Operator	Explanation
+	Adds two money objects
-	Subtracts two money objects
+=	Adds and updates the object
==	Checks whether two amounts are equal
<	Compares two amounts
<<	Displays money
>>	Reads money

4. Design and Analysis

1. Why does + return a new object?

The + operator creates a new Money object without changing the original objects.

2. Why does += return a reference?

The += operator modifies the current object. It returns *this to avoid creating another object.

Q5 – Overloading Increment, Comparison and Subscript Operators

Bloom's Level: K5 – Evaluate

Problem Context – Music Playlist and Track Timer

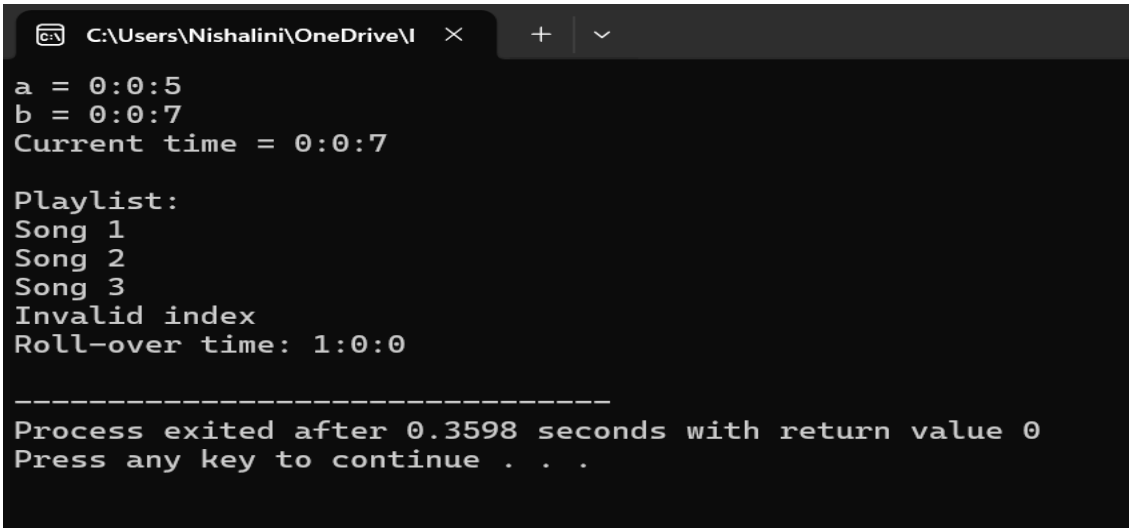
A music player keeps a playlist of tracks and a Time object for the current position. The player needs time++ and ++time to advance by one second, comparisons between times, and playlist[i] to reach the i-th track with a range check.

1.program



```
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;
class Time
{
public:
    int h, m, s;
    Time(int a=0, int b=0, int c=0)
    {
        h=a;
        m=b;
        s=c;
    }
    Time& operator++()
    {
        s++;
        if(s==60)
        {
            s=0;
            m++;
        }
        if(m==60)
        {
            m=0;
            h++;
        }
        return *this;
    }
    Time operator++(int)
    {
        Time temp=*this;
        return temp;
    }
    bool operator==(const Time &t) const
    {
        return h==t.h && m==t.m && s==t.s;
    }
    bool operator<(const Time &t) const
    {
        return h*3600+m*60+s < t.h*3600+t.m*60+t.s;
    }
    void display() const
    {
        cout<<h<<":"<<m<<":"<<s<<endl;
    }
};
class Playlist
{
    string songs[3]={"Song 1","Song 2","Song 3"};
public:
    string& operator[](int i)
    {
        if(i<0 || i>=3)
            throw out_of_range("Invalid index");
        return songs[i];
    }
    const string& operator[](int i) const
    {
        if(i<0 || i>=3)
            throw out_of_range("Invalid index");
        return songs[i];
    }
};
int main()
{
    Time t(0,0,5);
    Time a=t++;
    Time b=++t;
    cout<<"a = ";
    a.display();
    cout<<"b = ";
    b.display();
    cout<<"Current time = ";
    t.display();
    Playlist p;
    cout<<"\nPlaylist:\n";
    try
    {
        cout<<p[0]<<endl;
        cout<<p[1]<<endl;
        cout<<p[2]<<endl;
        cout<<p[-1]<<endl;
    }
    catch(const out_of_range &e)
    {
        cout<<e.what()<<endl;
    }
    Time x(0,59,59);
    ++x;
    cout<<"Roll-over time: ";
    x.display();
    return 0;
}
```

2.output



Design and Analysis

1. Prefix and Postfix: Prefix uses operator++() and postfix uses operator++(int).
2. Postfix Copy: Postfix returns the old value, so it creates a temporary copy.
3. Two [] Operators: Non-const allows modification; const allows only reading.
4. Invalid Index: Displays an error using out_of_range.

3. Predict the Output

```
Counter c{5};
Counter a = c++;
Counter b = ++c;
cout << a.v << ' ' << b.v << ' ' << c.v;
```

```
5 7 7
```

Explanation:

- Initially c = 5.
- c++ stores old value 5 in a and increments c to 6.
- ++c increments c to 7 and stores 7 in b.

4. Experimental Investigation

Prefix and Postfix Performance

Test 10,000,000 increments using -O0 and -O2.

Prefix is generally faster because postfix creates a copy. Actual time depends on the computer and compiler.

Invalid Index Test:

- playlist[-1] → Invalid index.
- playlist[size] → Invalid index.

Time Rollover:

59:59 becomes 1:00:00 (from 0 hours).

5. Alternative Concept and Complexity

Method	Description
operator[]	Access playlist

at(i)	Checks index and throws an error
Unchecked []	Accesses without checking

All methods take O(1) time. Checked access performs one extra comparison. For 10,000,000 accesses, measure the actual execution time to compare the difference.

6. Conclusion

Prefix ++i is preferred because it does not create an extra copy. Postfix i++ creates a temporary copy and may take more time.

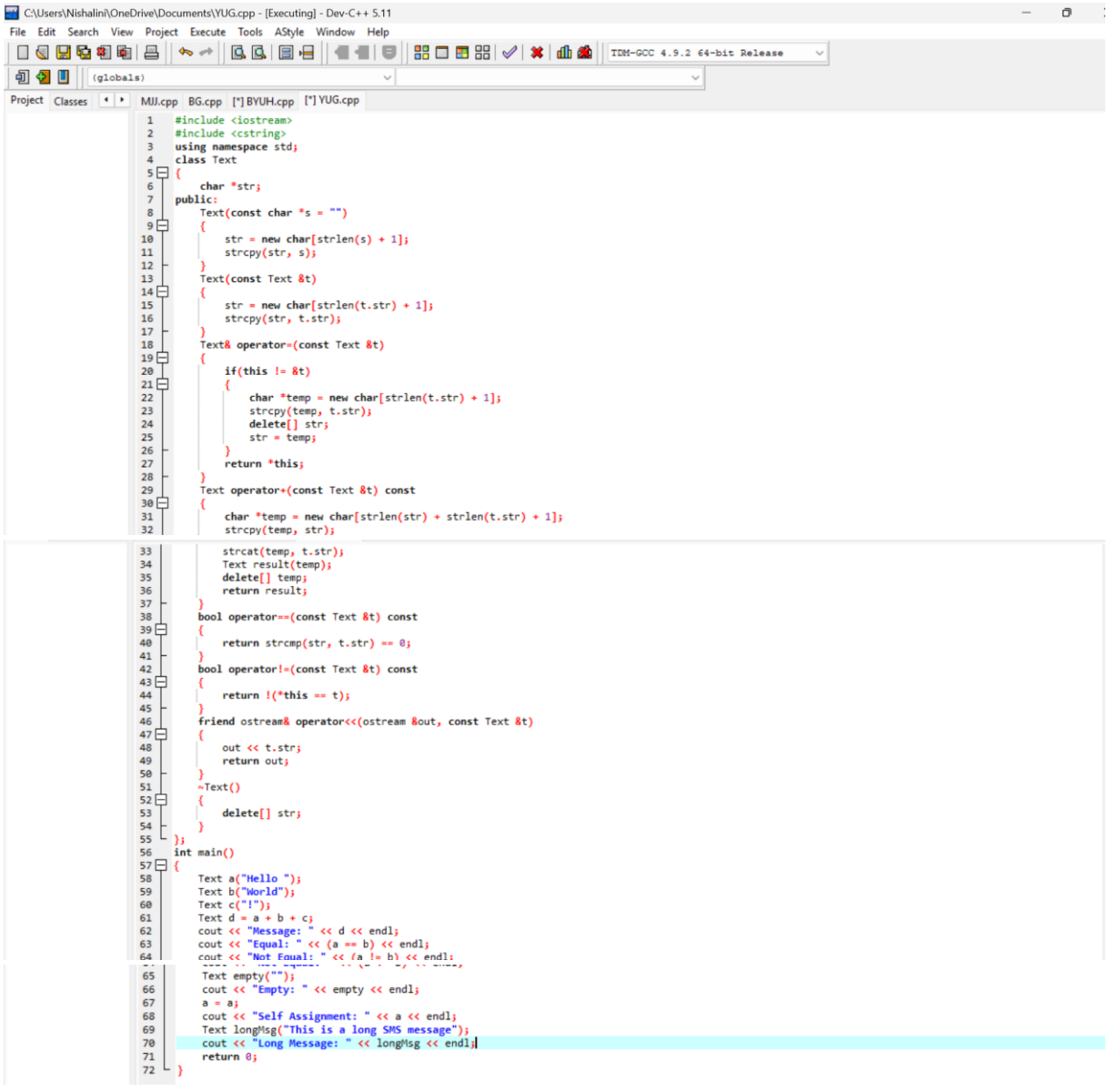
Q6 – The Rules of Operator Overloading

Bloom's Level: K5 – Evaluate

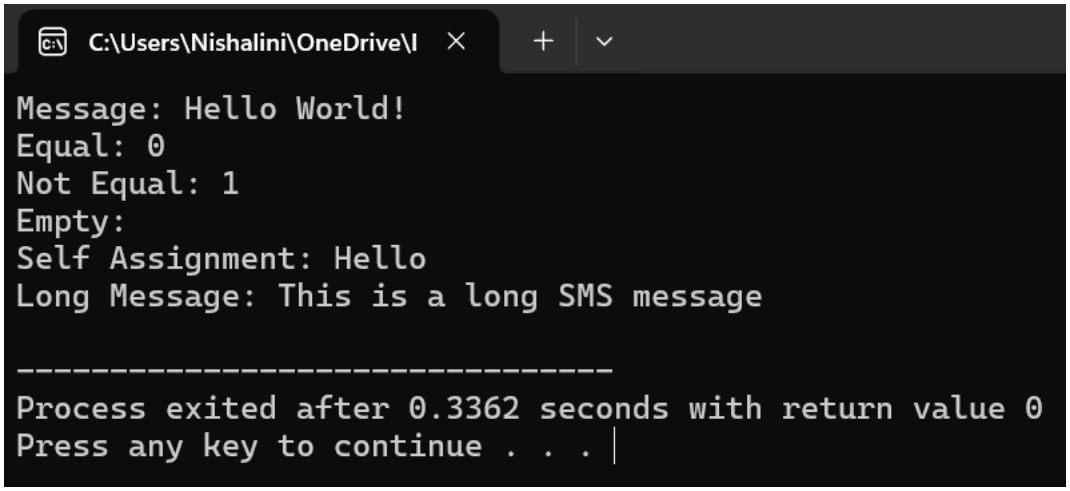
Problem Context – A Simple Text Class for an SMS Gateway

An SMS gateway uses its own Text class (holding a dynamically allocated character array) so that it can control memory carefully. It needs assignment, concatenation with +, comparison and printing.

1.PROGRAM



2.OUTPUT



3. Design and Analysis

1. Operators that cannot be overloaded
   - :: – Scope resolution
   - . – Member access
   - .* – Pointer to member
   - ?: – Conditional operator
   - sizeof – Size operator
2. Rules of operator overloading
   - Precedence cannot be changed.
   - Associativity cannot be changed.
   - Number of operands cannot be changed.
   - At least one operand must be a user-defined type.
3. Why must = be a member function?

Assignment operator = must be overloaded as a member function because it modifies the left-hand object.

4. When is friend used?

Friend functions are used when an operator needs access to private data but must be a non-member function. Example: <<.

Comparison with std::string

- Text uses manual memory allocation.
- std::string manages memory automatically.
- std::string is safer and easier to use.
- Test both with 100,000 concatenations and compare execution time.

4. Conclusion

Writing the Text class teaches dynamic memory allocation, copying, assignment and operator overloading. Custom classes are useful when special memory management or specific behaviour is required.

Submission Checklist

- Create the repository DSA01-<RegisterNumber>-Assignment3, one folder per question (Q1 … Q7), with the .cpp files, screenshots of every run and a README.md (name, register number, compiler g++ 13, compile commands, steps to run).
- Commit after each question (at least 7 commits) – a single upload will not be accepted.
- Run every program on your own machine, take screenshots, and replace the timings with your measurements. Run Q3 leak2.cpp under AddressSanitizer yourself (it could not be run here).
- Write the "Predict the Output" answers (Q3, Q5) BEFORE running the programs, then compare them with the actual output.
- Be ready for the viva: you must be able to explain and modify any program.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C++ source code for all questions
- Predictions, outputs, memory diagrams, sanitizer reports and timings

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, tests or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.